

HACKNEX

Internal Qualifier Round

Problem Statement Booklet

Presented by

Division of Computer Science and Engineering
Karunya Institute of Technology and Sciences

Generative AI · Computer Vision · Agentic AI · Applied ML

TABLE OF CONTENTS

S No.	Title	Page No.
1	Submission Guidelines	3
2	How to Submit	3
3	Rules and Instructions	4
4	Internal Qualifier Problem Statements	5-14

Submission Guidelines

Regardless of the problem statement you choose, your submission must demonstrate a **working solution** and clearly explain how it was built.

Your submission should include:

- **Working System** – A functional solution that addresses the chosen problem.
- **Source Code** – A Git repository containing the complete source code and a clear README with instructions to set up and run the system end-to-end.
- **Data Pipeline** – A clear demonstration of how input data is collected, processed, and passed through your system.
- **Core Model / Reasoning** – The central logic, model, or reasoning mechanism that powers your solution.
- **Evidence & Explanation** – Supporting evidence that demonstrates how your system works, such as citations, timestamps, confidence scores, intermediate outputs, or relevant code.
- **Sample Input & Output** – At least one representative example showing the system working from input to output.
- **Scope Note** – Clearly distinguish the minimum viable solution you implemented from any additional stretch goals or features you attempted.
- **Live Demonstration** – Demonstrate the working system during evaluation. A recorded demonstration may be used where permitted.

How to Submit

Submit your project through a **Git repository** that is public.

The repository must contain a **clear and complete README** explaining:

- What the project does
- Technologies, libraries, and models used
- How to install dependencies
- How to configure and run the system
- How to reproduce the demonstrated results

Your submission counts towards your participation, so be sure to submit the link by the end of your evaluation!

* A presentation (ppt) is not necessary!

Submission Link: <https://forms.gle/KGjkU5u66Va1MDhu5>

Rules and Instructions

- **Preparation is Allowed:** Problem statements have been shared in advance. Teams may use this time to understand the problem, research relevant technologies, plan their approach, and prepare their development environment.
- **AI Usage:** The use of AI tools is permitted, provided teams review, understand, and take responsibility for their submitted work.
- **Declare Your Resources:** Teams should clearly mention any external APIs, datasets, pre-trained models, open-source components, or other significant resources used in their project.
- **Demonstrate Your Work:** Teams should be able to explain their approach, demonstrate the working system, and answer questions about the implementation during evaluation.
- **Evaluation Integrity:** Teams are expected to work independently and maintain the integrity of the evaluation process. Any attempt to access or manipulate evaluation data or hidden test cases will not be permitted.
- **Team Conduct:** Participants are expected to maintain a respectful and collaborative environment with fellow participants, mentors, volunteers, and organizers.
- **Organizer's Decision:** The evaluation team's decision regarding qualification and judging will be final.

HNX26PSI01: Multimodal Document Intelligence

Generative AI · Vision-Language Models · RAG · Document AI

What You're Building

Create a system that can read mixed documents (PDFs with text, tables, charts, graphs, images, scanned pages) and answer questions about them. The system needs to find answers by looking at multiple types of content and tell you exactly where in the document it found that answer.

***Example:** If someone asks "Compare production efficiency between Q2 and Q4, identify the three biggest reasons for the change, and show me the proof" – your system must find and combine information from text, tables, AND charts to answer it.*

Key Rules

- Every answer must point to where it came from (which document, which page, which section). An answer with no source = no points.
- If you need a chart or table to answer, you can't just read the text version. Visual content must actually be analyzed as visual.

Scoring Metrics

- Accuracy on questions mixing text + tables + charts + images
- Is the evidence correct and do answers actually match the cited source?
- Can it find information across multiple documents?
- Can it do math and handle numbers correctly?
- Does it handle tricky documents (scanned pages, multiple columns, messy layouts)?
- Does it still work when documents are poor quality or damaged?

What to Build First

- Get a document collection (you'll need to find or create realistic mixed documents)
- Make sure it can answer text and table questions correctly with page-level sources
- **Advanced:** Handle chart reasoning and cross-document questions

HNX26PSI02: Video Understanding & Temporal Reasoning

Computer Vision · Vision-Language Models · Video AI · Temporal Reasoning

What You're Building

Build a system that watches videos and answers questions about what happened, when it happened, and the order of events. Your system needs to understand time — not just "what object is this" but "what happened first, then what, and how long between events."

Example Questions:

- *Which person entered the restricted area after the delivery truck arrived?*
- *How many times did the machine stop unexpectedly?*
- *What happened right before the safety alarm?*
- *Find every object that sat there untouched for more than 2 minutes.*

Key Rules

- Every answer must have a timestamp. No timestamp = no points.
- If the answer is right but the time is wrong, you get only half credit. Getting the when is as important as getting the what.

How You'll Be Judged

- Can it figure out order, cause & effect, and count things over time?
- Are the timestamps correct?
- Does it find events accurately?
- Can it track people/objects and know it's the same one even after they leave the frame or get hidden?
- Does it answer the question correctly?
- Does it handle long videos, when things block the view, or when the camera moves?

What to Build First

- Collect or record test videos (find open datasets or create your own)
- Start with simple "what happened and when" questions on short clips
- **Advanced:** Handle long videos, multiple events, camera angles changing

HNX26PSI03: AI-Powered Cyber Threat Intelligence

AI · Cybersecurity · Graph Analytics · Anomaly Detection

What You're Building

Security systems collect millions of logs from user logins, computers, applications, and networks. Each log by itself looks normal, but together they might show a coordinated attack. Build a system that connects these events across users, devices, IP addresses, and apps to reconstruct what an attacker did, step by step. Your output should show: timeline of the attack, who was involved, stages of the attack, proof for each stage, how risky it is, and what to do about it.

***Example:** A user logs in from an unusual location, then accesses files they've never touched, then copies them to a USB drive they plugged in. One event = suspicious. All three together = attack.*

Key Rules

- False positive limit: If your system raises too many false alarms on clean logs, your score gets capped. More alerts ≠ better score.
- Every stage of the attack must have proof. An unsupported claim = doesn't count.

How You'll Be Judged

- Can it link separate events into one attack chain in the right order?
- Does it correctly identify attacks without false alarms?
- Does it catch all the real attacks?
- Can it connect users, devices, IPs, and apps correctly?
- Does it stay quiet on clean, benign logs?
- Is the timeline accurate?
- Does it explain why it thinks something is an attack?

What to Build First

- Get or create sample security logs (CTF datasets, DARPA datasets, or simulate your own)
- Start by detecting one known multi-stage attack
- **Advanced:** Detect slow, hidden, or never-before-seen attacks

HNX26PSI04: Real-Time Financial Fraud Intelligence

FinTech · Graph ML · Anomaly Detection · AI

What You're Building

Fraud doesn't always hide in one suspicious transaction — it hides in patterns across many accounts, devices, stores, and payment channels. Build a system that learns normal behavior, spots when something is off, and finds coordinated fraud rings. Your output should show: risk score for each transaction, risk score for each account, which accounts are suspicious together, what pattern you found, a diagram showing connections, and what action to take.

***Example:** Ten accounts from different people, different locations, different devices — but they all buy the same unusual items in the same sequence, then move money to the same place. No single transaction looks bad. The pattern says fraud.*

Key Rules

- False positive limit: If you flag too many legitimate transactions as fraud, your score gets capped. Precision matters.
- Every risk score must have an explanation. A score with no reasoning = no evidence points.

How You'll Be Judged

- Can it find fraud rings (multiple accounts acting together)?
- Does it catch fraud without false alarms?
- Does it catch most of the real fraud?
- Does it rank transactions and accounts by risk correctly?
- Does it avoid flagging legitimate high-value transactions?
- Can it spot unusual behavior compared to normal?
- Does it explain why something looks like fraud?

What to Build First

- Create or find transaction data (IEEE fraud dataset, Kaggle datasets, or simulate your own)
- Start by detecting one fraud ring in the network
- Advanced: Handle new patterns and accounts that change behavior

HNX26PSI05: Multimodal Medical Image Intelligence

Computer Vision · Medical AI · Multimodal AI

What You're Building

Build an AI system that doctors can use as a second opinion (not a replacement). It reads medical images (X-rays, CT scans, etc.), combines that with patient notes or test results, and gives an assessment. It must point to exactly where in the image it sees something unusual, explain what it found, and tell you how confident it is. It's a helper for doctors, not a replacement.

Key Idea: A finding with no location in the image or no supporting clinical notes is a hallucination and fails that case.

Key Rules

- Every finding must be backed up. Either point to a region in the image OR cite information from patient notes. Unsupported findings = hallucinations = fail.
- Must include confidence levels. "I'm 95% sure" vs "I'm 30% sure" — don't say everything with certainty.
- Must present as a helper, not a doctor. "Doctor, consider this finding..." not "The patient has..."

How You'll Be Judged

- Can it spot and classify abnormalities correctly?
- Can it point to the exact region in the image?
- Does it use both images AND patient context together?
- Can it segment/outline problem areas (advanced)?
- Are confidence levels realistic?
- Does it explain findings with evidence?
- Does it handle poor-quality images?

What to Build First

- Use public medical datasets (ImageNet Medical, Kaggle, or similar)
- Start with one type of abnormality on single images with a heatmap
- Advanced: Segment regions, combine image + patient notes

HNX26PSI06: Generative Computer Vision & Scene Reconstruction

Generative AI · Computer Vision · Diffusion Models

What You're Building

Generative AI can create realistic content, but keeping things consistent is hard. Build a system that fills in missing, damaged, or blurry parts of images (or videos) while keeping everything else the same — same objects, same positions, same perspective, same lighting. If working with video, frames must look consistent frame-to-frame without flickering.

Example: Remove a person from a photo and fill the gap without changing the background. Or fix a corrupted video frame without making the actor flicker.

Key Rules

- Object identity, count, and position must not change in undamaged regions. A pretty fill-in doesn't count if it moved something that shouldn't move.
- For video: no flickering or identity drift between frames. Temporal consistency is critical.

How You'll Be Judged

- Are position, angle, and geometry preserved?
- Does the filled content look realistic?
- Are objects the same identity (not replaced)?
- Does it look good visually?
- For video: no flicker or jumping (temporal consistency)?
- Does it work on hard cases (shadows, reflections, complex scenes)?

What to Build First

- Use public datasets (Places, COCO, or similar) with masks
- Start with masked regions on single images
- Advanced: Handle video with frame-to-frame consistency

HNX26PSI07: Autonomous Vision & Behaviour Understanding

Computer Vision · Action Recognition · Object Tracking · Behaviour Analysis

What You're Building

Finding objects in video is easy — understanding what they're doing is hard. Build a vision system that finds people or objects, tracks them (knows it's the same one even if it moves), and understands their behavior. Spot if behavior is normal or unusual, and surface meaningful events (not just "there's a person").

Pick a scenario: workplace safety, retail, factory, campus, traffic, or crowds.

Example: A person walking around a warehouse is normal. A person standing still in one spot for 10 minutes is unusual.

Key Rules

- Can't just list detections. Must explain what they're doing, not just that they exist.
- Every flag for unusual behavior must point to which entity and when. "Something weird happened" doesn't count — say who and when.

How You'll Be Judged

- Can it recognize what people/objects are doing?
- Does it spot meaningful events?
- Can it tell normal from abnormal behavior?
- Does it track the same object across the video?
- Does it find objects accurately?
- Are events detected at the right time?

What to Build First

- Collect videos for your chosen scenario
- Detect and track objects, classify normal vs one abnormal behavior
- Advanced: Handle multiple behavior types, general anomaly detection

HNX26PSI08: Proof-Carrying Data Analyst (Agentic GenAI)

Agentic GenAI · Data Analytics · Code Generation · Verification

What You're Building

Build an AI agent that answers questions about messy, real-world data in multiple tables and documents. The key: every number must come with working code that someone else can run to check your math. If the code doesn't run or gives a different answer, you fail. The agent must also know when a question can't be answered reliably — a confident wrong answer is worse than saying "I can't determine this from the data."

Traps to watch for:

- Units don't match (\$/€)
- Dates are ambiguous
- Rows are duplicated
- Tables contradict each other
- Data is missing
- Question has no valid answer
- Question is designed to trick you

Key Rules

- Every number must have re-runnable code. Verifier runs your code, gets your number, or you score 0.
- Wrong + confident explanation scores worse than "I can't determine this" + good reason. Know when to say no.

How You'll Be Judged

- Is the answer correct?
- Can someone else run the code and get the same answer?
- Is the math correct?
- Does it correctly refuse unanswerable questions?
- Does it spot ambiguity (date ranges, currency, etc.)?
- Does it handle messy data (dupes, missing values, unit mismatches)?
- Does the code actually run?

What to Build First

- Create or find messy multi-table data
- Answer basic questions with working code
- Advanced: Handle adversarial data, trap questions, ambiguity

HNX26PSI09: AI Software Engineering Agent

Generative AI · Coding Agents · Software Engineering

What You're Building

Real codebases are huge (thousands of lines across frontend, backend, database, APIs, tests, config, docs). Build an agent that:

1. Reads an existing codebase
2. Understands how it works
3. Makes a requested change or fixes a bug
4. Makes sure it doesn't break anything that was working before

Example: "Add X feature while keeping Y workflow working" — agent finds the right files, makes changes, runs tests, confirms nothing broke.

Key Rules

- Existing tests must still pass. If you break something that was working, you lose major points.
- No making up APIs or imports that don't exist. Must use real code.

How You'll Be Judged

- Do hidden tests pass (does your change work)? 30 points
- Does the change work (visible acceptance tests)?
- Did you break anything that was working?
- Did you understand the codebase and change the right parts?
- Is the code clean and minimal?
- Can you explain the bug?
- Is your test code good?

What to Build First

- Use a public repository (GitHub, GitLab, etc.)
- Fix one documented bug with working tests
- Advanced: Multi-file changes, feature additions, non-obvious bugs

HNX26PSI10: Multimodal Deepfake & Digital Forensics

Computer Vision · Generative AI · Audio AI · Digital Forensics

What You're Building

AI can now create fake videos and audio that look real. Build a forensics system that watches images, videos, and/or audio to decide: is this real or fake? Give it an authenticity score (like "80% likely real"), and explain what signals made you decide. Don't rely on just one artifact — good fakes can hide one mistake, but multiple signals together tell the story.

What might be fake:

- Face swaps (one face on someone else's body)
- Completely synthetic faces (don't exist)
- Lip-sync manipulation (mouth doesn't match audio)
- Voice cloning (AI-generated speech)
- Image manipulation
- Re-encoding and compression

Key Rules

- Must work on techniques you haven't seen before. Good on known fakes = capped score. Must generalize.
- Every score must have an explanation. Just a score with no evidence = lose points.

How You'll Be Judged

- Does it detect types of fakes it never saw during training?
- Accuracy on known and unknown fakes?
- Can it point to where the manipulation is?
- Does it catch fakes without flagging real content?
- Does it handle compressed/cropped/re-encoded versions?
- Does audio match video (if both present)?
- Does it explain the signals behind the decision?

What to Build First

- Find authentic and fake content (FaceForensics++, DFDC, or create your own)
- Detect and score one modality
- Advanced: Cross-modal consistency, compression robustness